Phys and Lean: A Complementary Architecture for Trustworthy AI-Assisted Research

Addressing garbage-in, garbage-out across the full journey from a scientific question to a justified claim

Technical position paper and collaboration proposal — Isagani Mendoza (Pithom Labs)

Status: the Phys–Lean interoperability contract and staged pilot described here are proposals. This paper does not claim that a native bridge, shared schema, automated exporter, or integrated validation pipeline has already been implemented.

Executive summary

Phys and Lean address different but connected failure modes in AI-assisted reasoning. Lean 4 is a mature programming language and interactive theorem prover whose small kernel checks proof terms against formal propositions, definitions, and axioms. Its mathematical ecosystem, especially Mathlib, provides a substantial body of reusable formal mathematics. That makes Lean exceptionally valuable when the goal is to establish that a proof follows from a precisely stated formal proposition under declared foundations.

Phys is being developed around a complementary question: before we ask a proof checker to certify an argument, how do we make sure the problem, source claim, physical assumptions, mathematical representation, interpretation, and claimed evidence have been captured faithfully? Phys separates durable artifact identity (PhysArtifact), theory-neutral mathematical representation (PhysMath), physical-framework semantics (PhysLib and the `phys-*` packages), and derivation workflow, provenance, and evidence discipline (PhysTrace and its specifications). The architecture explicitly forbids inferring physical or empirical truth merely from mathematical consistency, successful replay, projection, or contract validation.

The proposed relationship is therefore cooperative, not competitive. A useful shorthand is: **Phys helps investigate the fidelity of what goes in; Lean can rigorously check what follows from the formal statement that went in.** This is a division of emphasis, not an absolute partition. Lean can expose contradictions in a formalized problem, and Phys's source mapping, capability audits, and tests can catch some output-side mistakes. Neither system alone can establish that an AI-selected hypothesis is the right question, that a formalization captures a research paper's intended meaning, or that a mathematically coherent theory describes Nature.

The difficult problem is bigger than theorem proving. Humans and AI agents must formulate useful hypotheses, translate informal ideas into precise objects, select appropriate assumptions and regimes, discover candidate implications, distinguish derivation from empirical confirmation, and revise their views in response to evidence. Some of this work needs human judgement and accountability; much of the search, bookkeeping, checking, counterexample generation, and routine proof exploration can be increasingly autonomous. The architecture must support both without confusing automation with authority.

This paper proposes a staged Phys–Lean collaboration: use Phys to create source-grounded, assumption-explicit, provenance-bearing research packages; review candidate formal statements for fidelity; translate suitable mathematical claims into Lean; mechanically check proofs and audit axioms and dependencies; then preserve both the formal result and its scientific context. Such an integration is a proposal, not a claim that a Phys–Lean bridge already exists. The opportunity is to combine research-input discipline with proof-level verification and ultimately with experimental evidence, while keeping humans in the loop where interpretation and responsibility genuinely matter.

1. The general problem: correctness is a pipeline, not a single checkbox

Imagine an AI assistant reading a paper, extracting a theorem, translating it into a formal language, producing a proof, and reporting success. A proof checker may correctly verify the proof. Yet a major scientific failure may remain: perhaps the assistant extracted the wrong equation, dropped a condition, confused two quantities, formalized an author's conjecture as a theorem, or silently replaced the intended claim with an easier one. The proof can be impeccable while the research result is wrong for the purpose that motivated it.

Now imagine the opposite situation. A researcher states the intended result accurately, but the proof is invalid or the symbolic manipulation is wrong. Here the problem is not primarily source fidelity; it is the validity of the downstream reasoning. A proof assistant such as Lean is built to address this second class of failure for propositions expressed in its formal logic.

These examples illustrate why a scientific workflow needs more than one kind of correctness. At least five layers should be distinguished:

1. **Question correctness:** Is this a meaningful, well-scoped question? Does it target a real gap or a worthwhile explanatory problem?
2. **Source and interpretation fidelity:** Did we capture what a paper, experiment, or researcher actually claims, including its definitions, assumptions, conventions, and limitations?
3. **Formalization fidelity:** Does the formal object express the intended informal or physical claim rather than a subtly different, weaker, stronger, circular, or vacuous one?

4. **Derivational correctness:** Does the conclusion follow from the stated premises by valid mathematical steps? Were the operations actually performed and are their proof obligations discharged?
5. **Empirical adequacy:** Does the model agree with evidence about the physical world, within the regime and uncertainty where it is meant to apply?

Each layer has different tests and different sources of authority. A unit test can detect an implementation regression but cannot decide whether a theory is physically adequate. A source citation can establish where a formula appeared but not that the formula is true. A Lean-checked theorem can establish a formal consequence but cannot, by itself, certify that the formal proposition captures the author's intent. An experiment can constrain a model but still rely on measurement assumptions, calibration, and statistical inference.

The larger aim is not to make one system the universal judge of truth. It is to preserve the connections between these layers, so that a user can trace a result from the question and source, through assumptions and formal representation, into derivation evidence, and eventually to empirical tests when such evidence exists.

2. Garbage in, garbage out—and where the phrase is incomplete

“Garbage in, garbage out” is often used as a warning about data quality. In research automation it is broader. The bad input might not be a corrupt file or low-quality dataset. It might be an ill-posed question, a false premise, a misunderstood definition, an omitted boundary condition, the wrong physical regime, a misleading source excerpt, or a formal theorem that no longer says what the human thought it said.

We can separate the problem into two broad sides.

The input-side problem

Before reasoning begins, an agent needs a faithful representation of the task:

- Which source and version are authoritative?
- What exactly is being claimed, and where in the source is it stated?
- Which definitions determine the meaning of the symbols?
- Which assumptions are explicitly stated, inherited from a framework, inferred by the analyst, or still unresolved?
- What regime, domain, conventions, and boundary conditions apply?
- Is a displayed equation a premise, a definition, a derived step, a heuristic, an approximation, or a claim cited from elsewhere?
- Which transformations are supported by the available software, and which remain a human derivation or an unimplemented capability?

- What would count as a useful counterexample, a falsifying condition, or a competing explanation?

These are not clerical details. They determine what problem is being solved. If they are wrong, a downstream system may confidently produce a result that is perfectly correct relative to a question nobody intended to ask.

The output-side problem

Once a precise statement exists, the work still has to be done correctly. A system may invent a proof step, use an invalid algebraic rewrite, omit a logical case, mishandle a domain condition, or produce a plausible but unsound derivation. This is where proof checking, executable transformation rules, independent expected results, adversarial tests, and other forms of verification matter.

Why a proof checker cannot solve the whole input problem

Lean's official reference describes Lean as a theorem prover based on dependent type theory, with a small kernel that checks proof terms. Tactics generate terms that are checked by the kernel, so ordinary mistakes in tactic automation do not by themselves invalidate the kernel's checking role [1]. This is a powerful guarantee—but it is conditional. Lean's own guidance explicitly distinguishes whether a theorem has a valid proof from what the theorem statement means. Confidence in a checked theorem depends, among other things, on whether the formal statement corresponds to the intended informal meaning and whether its axioms and imported dependencies are appropriate [2].

A proof checker can verify `A → B` even if the research question should have been `C → B`. It can verify a theorem under an assumption that no one should have accepted. It can verify a statement whose definitions encode a proxy for the intended physical concept. It can prove a result about an elegant model without showing that the model describes a real system. These are not defects in proof checking. They are consequences of asking a proof checker a question that it is designed to answer.

Why input discipline alone is insufficient

The reverse is equally important. A perfect source map and a beautifully recorded list of assumptions do not guarantee valid reasoning. The implementation may still perform the wrong transformation or fail to establish the final result. Source fidelity and proof validity are different obligations. A serious research platform needs both—and should refuse to collapse them into one “verified” badge.

For Phys, this is the central framing: it is being designed to make the formulation and provenance of research claims explicit and auditable. It does not eliminate bad inputs automatically. It makes relevant inputs, choices, gaps, and evidence visible enough to review,

test, and challenge. Lean then offers a mature way to check suitable formal consequences once the statement and its foundations have been responsibly established.

3. Lean's role and its strengths

Lean 4 is both a functional programming language and an interactive theorem prover. Its core logic is based on dependent type theory. Proofs are represented as terms whose types express propositions, and the kernel checks those terms. The tactic language, elaborator, and supporting automation help users construct proofs, but the kernel's acceptance of the resulting proof term is the principal assurance that the proof follows within the formal system [1].

Mathlib, the community-maintained mathematical library for Lean 4, adds definitions, theorems, tactics, and mathematical infrastructure across a broad range of subjects. Lean's current Mathlib overview describes a library with more than two million lines of formalized mathematics, used in research mathematics and software-verification applications [3]. This accumulated work is one of Lean's most important strengths: a formalization can often reuse established definitions, lemmas, abstractions, and proof techniques rather than starting from zero.

Lean's most relevant strengths for a broader scientific workflow are:

- **Mechanical proof checking.** For a suitable formal proposition, Lean can reject incomplete or invalid proof terms rather than relying on an AI's verbal confidence.
- **Explicit assumptions and dependencies.** The type of a theorem and the constants, definitions, axioms, and imported results it uses can be inspected. Lean provides tools such as `#print axioms` to audit the axioms on which a theorem depends [2].
- **Composable formal knowledge.** Mathlib lets later work build on a large, reviewed, reusable base of mathematical results [3].
- **A disciplined development loop.** Theorem statements, proof scripts, compilation, CI, and library review make progress reproducible and regressions detectable.
- **Useful AI-agent interface.** An AI can propose definitions or proof steps, iterate on compiler feedback, search for lemmas, and autonomously pursue proof attempts while Lean checks the resulting artifact.

The most important guarantee should be stated accurately: Lean can check that a proof of a particular formal statement follows from the foundations, definitions, theorems, and axioms used by the formalization. This is far stronger than asking a language model whether its own answer looks right.

But it is not the same as validating the entire scientific claim. The theorem statement might not preserve the source's meaning; the imported definitions might encode a disputed interpretation; custom axioms may be necessary; and the mathematical model may still be inadequate as a description of Nature. Lean's own proof-validation documentation calls attention to the gap between a theorem having a valid proof and the meaning of the theorem statement [2]. In other

words, Lean is especially strong on the formal consequence side of the problem, while its users and surrounding workflow remain responsible for the significance and intended meaning of the proposition being proved.

4. Phys's role and its strengths

Phys is a layered infrastructure effort for physics-related mathematical representation, research provenance, and derivation workflows. Its architecture deliberately distinguishes identity, mathematical structure, physical-framework semantics, and workflow evidence rather than placing all of them into one generic object or one universal solver.

PhysArtifact: identity without semantic overreach

PhysArtifact provides canonical identity and reference mechanics for durable artifacts. It helps answer “which exact artifact are we talking about?” It is not a mathematics checker, a trust authority, a physical ontology, or a scientific verdict engine.

PhysMath: a bounded, theory-neutral mathematical layer

PhysMath represents mathematical objects and supported symbolic transformations without taking a position on the physical interpretation by itself. It has an explicit executable rule set and tests. The existence of a representation does not imply there is a rule that executes the desired operation: some content can be stored or represented without being executable by the current engine. The live rule registry and tests—not a high-level description or a capability-map label—determine what the implementation actually supports.

This restraint is important. PhysMath is not intended to become an all-purpose theorem prover or an unlimited symbolic algebra engine. The project architecture explicitly excludes a generic theorem prover from PhysMath, keeps theory-specific physical semantics outside its core, and prohibits treating replay or successful projection as proof of scientific truth [4]. Lean is therefore not something PhysMath should try to reproduce internally. A future integration should invoke Lean's proof capability across a well-defined interface rather than blur the responsibilities of both projects.

PhysLib and `phys-*`: mathematical structure with physical meaning

PhysLib's common core and theory-specific packages represent physical-framework semantics where they belong. A mathematical expression by itself does not determine whether it is an equation of motion, a constraint, a definition, an approximation, or a statement with a particular interpretation. Nor should General Relativity's assumptions be forced onto quantum mechanics or Bohmian mechanics simply because one package happens to have useful code.

The project uses this separation to make interpretations and assumptions more explicit. It also records whether a claim is established, incomplete, cited, open, or otherwise limited according

to the governing status vocabulary. This helps prevent a research agent from turning an unverified claim into a proved theorem just because it has been encoded in a convenient structure.

PhysTrace: source fidelity, method, and evidence discipline

PhysTrace's derivation-specification guidance calls for exact source identity, an explicit map from quantities to source equations, assumption and condition tracking, capability audits, independent expected results, anti-golden or mutation tests, and explicit reporting of what was or was not executed [5]. It emphasizes distinctions among constructing an expression, validating its structure, executing a rule, replaying a trace, proving an algebraic equality, interpreting a physical result, and obtaining empirical support.

The two documented package contracts serve different purposes. Type A is for formalizing a theorem, hypothesis, derivation, or paper-scale result under the exact established formalization contract. Type B is for solving and documenting a specified benchmark or textbook problem. The Type-B protocol requires every claimed library reuse to cite a real public API, a code path that calls it, and a test that actually exercises it; imports alone do not count. Unsupported computation must be attributed to solver-local work rather than to Phys. A Type-B label also cannot be used to evade Type-A requirements when an artifact claims paper-scale formalization [6].

Phys's strengths are therefore complementary to Lean's. It is intentionally attentive to the research context around a derivation: provenance, physical interpretation, assumptions, source fidelity, capability boundaries, and honest reporting of gaps. Its weaknesses are equally clear: it is an earlier and more bounded infrastructure effort, not a mature general proof assistant; its current executable symbolic operations are limited; its workflow contracts require continued implementation and enforcement; and it cannot guarantee the correctness of every source interpretation or hypothesis simply by recording them.

5. Similarities and differences: a division of emphasis

Both projects aim to make technical claims more explicit, reusable, inspectable, and less dependent on trust in a single person or model. Both benefit from precise representations, tests, shared libraries, reproducibility, and disciplined contributions. Both can be used in AI-agent workflows, and both can reduce classes of errors that are hard to spot in fluent natural-language output.

Their main difference is the layer of the problem each makes central.

Dimension	Phys infrastructure	Lean 4 and Mathlib	Complementary use
Primary emphasis	Faithful representation of source-grounded physical/mathematical research, assumptions, interpretation, provenance, capability, and evidence status	Formal propositions, definitions, proof terms, and mechanical verification of mathematical consequences	Phys prepares and contextualizes the claim; Lean checks suitable formal consequences
Main input	Paper, equation, physical framework, assumptions, regime, derivation, source locations, benchmark question	Formal statement, definitions, imported theorems, axioms, and proof candidate	Connect the source-level claim to a reviewed formal statement
Core guarantee	What is represented, what capability is genuinely exercised, what source/evidence is attached, and which gaps are declared—according to current contracts and tests	The kernel accepts a proof of the formal statement under its declared logical foundations and dependencies	Report both evidence layers separately
Typical failure it helps expose	Wrong source mapping, dropped assumptions, invented operations, import-only library use, inflated status, undocumented unsupported steps	Invalid proof term, incomplete proof, failed logical step, some inconsistencies or impossible goals under the formal statement	Use both to find different classes of failure
Physical meaning	`phys-*` packages carry framework-specific semantics and	Can formalize physical definitions and theorems, but the meaning comes	Phys supplies explicit physical context and source links; Lean

Dimension	Phys infrastructure	Lean 4 and Mathlib	Complementary use
	interpretation commitments	from the definitions and statements encoded by the user	checks the selected formal content
Proof strength	A Phys replay or conformance pass is not automatically a theorem proof	Formal proof checking is a central native capability	Do not describe replay and proof checking as interchangeable
Empirical truth	Not automatically established by representation, replay, or a passing test	Not automatically established by a mathematical proof alone	Link to experimental evidence in a separate evidence layer
Maturity and ecosystem	Focused, bounded, actively developing physics infrastructure and protocols	Mature proof assistant with a large mathematics ecosystem	Reuse Lean; do not rebuild its kernel or Mathlib inside Phys
Current numerical scope	Numerical solvers and numerical integration are outside this representation-layer scope	Lean can express and prove properties about numerical algorithms; it is not automatically an empirical data pipeline or a substitute for all numerical computation	Keep numeric computation, symbolic representation, proof, and measurement as distinct services

This comparison should not be read as saying that Lean can only check outputs or that Phys only checks inputs. Lean's formalization and theorem statement are part of its trust boundary. Phys has tests and replay mechanisms that catch downstream mistakes. The distinction is a useful architectural allocation of responsibilities, not a claim that the systems never overlap.

6. The hardest problem is hypothesis formulation

In a frontier research workflow, the first difficult step is often not proving a theorem. It is deciding what theorem would be worth proving, what assumptions it should use, what mathematical

structure captures the physical situation, and what evidence could distinguish the hypothesis from alternatives.

A hypothesis may be too vague to test, too broad to be true, too narrow to be interesting, or circular because the conclusion is hidden inside the premises. A model may contain a parameter that was introduced for convenience but later mistaken for a physical entity. A formal statement may omit a critical regime condition. A proposed relationship may fit known observations yet make no novel prediction. A claim may be mathematically consistent but empirically underdetermined: several competing models may explain the same available data.

No proof assistant can decide, solely from a successful proof, whether the world would have been better explained by a different question. Yet without formalization, source discipline, and verification, even a good hypothesis can be lost in a mass of opaque AI-generated text.

The task is therefore to build a workflow that supports the entire path:

1. **Frame the question.** What phenomenon, discrepancy, or conceptual gap are we trying to understand? What would make the answer useful?
2. **Inspect the state of knowledge.** Which sources, results, and assumptions already exist? Where does the literature establish a result, merely suggest it, or leave a gap?
3. **Generate candidate hypotheses.** What alternative explanations or mathematical structures could address the question? Which predictions differ between them?
4. **Make the candidates precise.** Define the quantities, regimes, assumptions, and target statement. Where ambiguity cannot yet be removed, preserve it as an open choice rather than guessing silently.
5. **Audit source and formalization fidelity.** Check that the structured representation and candidate Lean statement really express the intended claim.
6. **Derive consequences.** Use supported symbolic operations, paper derivations, domain-specific tools, and Lean proofs as appropriate. Record unproved or unsupported steps.
7. **Search for failure.** Construct counterexamples, vary assumptions, check limiting cases, challenge the hypotheses with competing models, and test the implementation's claimed invariants.
8. **Compare against the world.** Identify what empirical evidence exists and which new observations could discriminate among the surviving hypotheses.
9. **Revise and preserve lineage.** Record which assumption or result changed, why it changed, and which previous conclusion it affects.

Phys and Lean contribute to several of these stages, but neither currently supplies the whole research program. Phys has a particular role in maintaining the relationship between sources, representations, assumptions, physical frameworks, and evidence. Lean can establish formal consequences once a candidate statement is precise enough to formalize. AI agents can

explore the intermediate search space—provided the system does not mistake the number of generated candidates for scientific progress.

7. Human-in-the-loop: where judgement should remain

Human review should not mean that a person manually approves every algebraic simplification, proof-search step, or file operation. That would waste the strengths of automation and make the system unusable. Nor should autonomy mean that an agent may silently choose the scientific meaning of an ambiguous claim, promote a result to empirical truth, or redefine the target until its proof succeeds.

A sensible design uses **risk-weighted human gates**.

Decisions that need human accountability

Humans should own or explicitly approve decisions such as:

- whether a question is scientifically or socially important enough to pursue;
- what the intended meaning of a source passage or physical concept is when the paper is ambiguous;
- which assumptions are justified by the theory, source, or empirical context;
- whether a proposed formal statement faithfully captures the informal claim;
- whether a new definition is scientifically meaningful rather than merely convenient for proof search;
- whether a model's interpretation is warranted and whether its limitations have been represented fairly;
- whether empirical evidence is strong enough to change a scientific claim's status;
- whether a high-impact conclusion should be communicated publicly, and with what caveats.

These decisions have contextual, interpretive, and often institutional consequences. A tool can prepare evidence and highlight inconsistencies, but assigning accountability still matters.

Tasks that can and should become autonomous

With bounded permissions and a durable audit trail, agents can autonomously:

- extract candidate equations and build a source map for human confirmation;
- enumerate symbols, definitions, parameter dependencies, and conditions;
- search the existing Phys and Lean APIs for relevant representations and lemmas;
- propose multiple formalizations rather than committing to the first plausible one;
- run parsers, schema validators, tests, builds, and proof-checking commands;
- attempt multiple proof strategies and report why attempts failed;

- generate counterexamples to candidate claims where tools permit;
- test boundary cases, limiting cases, sign choices, units metadata, and alternative assumptions;
- perform controlled mutation testing to verify that a checker rejects known bad inputs;
- identify proof dependencies, axioms, imported modules, and unresolved gaps;
- compare two formalizations and flag where assumptions, quantifiers, or definitions differ;
- compile a review bundle that a human can inspect efficiently.

An agent should be allowed to discover that it cannot proceed. That is productive information. It should not invent a capability or weaken the claim simply to maintain a successful-looking run. If the formalization cannot be represented, the workflow should preserve the gap, propose the smallest justified extension, and request authorization where architecture or scientific commitments would change.

The right boundary is controlled autonomy, not autonomy versus humans

A mature system will be highly autonomous in exploration and conservative in authority. It can run thousands of proof attempts, test variations, maintain a hypothesis tree, and uncover contradictions without requiring a person to supervise each micro-step. But it should not treat its own search results as self-authenticating. Major changes in interpretation, premise set, model identity, or evidence status should be explicit events with review requirements.

A practical distinction is: **agents may propose and investigate; formal checkers may certify bounded formal properties; the research protocol records what happened; humans adjudicate meaning and significance; empirical methods constrain claims about Nature.** Those roles can evolve as tools improve, but the evidential boundary should remain visible.

8. Autoformalization: translating language into logic without changing the claim

Autoformalization is the process of translating informal mathematics—or, more broadly, scientific reasoning—into a formal language that a computer can check. It is essential for combining natural-language research with proof assistants. It is also one of the most fragile points in the pipeline, because a successful translation must preserve meaning, not merely compile.

Consider a sentence such as “the distribution is preserved over time.” A physicist may mean a specific probability density, with a particular measure, dynamical law, boundary condition, interpretation, and regime. A formalization that proves preservation of a different distribution, assumes the desired conservation property, or quantifies only over an easier subcase may be syntactically valid and logically proved while missing the scientific claim.

Several failure modes deserve separate attention:

- **Lexical ambiguity:** the same word or symbol is used in multiple senses.
- **Hidden assumptions:** the text relies on conventions or domain conditions that are not repeated near the equation.
- **Quantifier drift:** “for every” becomes “there exists,” or a universal claim becomes a claim about a special case.
- **Scope drift:** a statement that holds in a specific physical regime becomes a general theorem, or the reverse.
- **Definition drift:** the formal definition is easier to reason about but no longer captures the source concept.
- **Vacuity or circularity:** the statement becomes provable because the assumptions already contain the desired result, the domain is empty, or the definitions make the claim trivial.
- **Proxy substitution:** the formalized variable is an approximation or convenient surrogate for the physical quantity but is described as the original quantity.
- **Proof-driven target rewriting:** the agent weakens or rewrites the proposition until Lean accepts it, then reports that it proved the original claim.
- **Dependency opacity:** an apparent result relies on an unreviewed axiom, placeholder, imported theorem, or definition with hidden assumptions.
- **Back-translation illusion:** translating formal code back into natural language may expose mismatches, but a fluent paraphrase is not a complete semantic equivalence proof.

This is an active research area, not a solved preprocessing step. An EMNLP 2025 study developed better evaluation measures and research-level benchmarks for statement autoformalization; in its experiments, current approaches achieved up to 45.1% accuracy on undergraduate-mathematics benchmarks and struggled with research-level content without sufficient context [7]. That number is specific to the study's data and evaluations, not a universal score for every system or task. More recent work published in June 2026 reports an agentic framework that formalized samples of PutnamBench problems and main theorems from five STOC papers, with the paper noting human expert validation of the paper formalizations [8]. This progress is encouraging, and the explicit role of expert validation is instructive: the more ambitious the formalization, the more important it is to verify that the formal statement is the intended one.

A stronger workflow for autoformalization

Phys can contribute by making source fidelity an artifact that is maintained, not a sentence in a prompt. For each claim, the workflow can record the exact source/version and location; the quantity-to-equation map; the definitions and regimes; assumption provenance; a source-faithfulness review; candidate formal statements; known alternative readings; and the status of each unresolved ambiguity.

Lean then checks whether the chosen statement has a proof under its specified foundation. Before accepting the result, the workflow should compare the informal source claim and the formal statement line by line, inspect imported dependencies and axioms, and test nearby variants that might reveal that the statement was accidentally weakened or made vacuous. Where possible, two independent formalization attempts and a back-translation can expose drift—but these are diagnostics, not substitutes for expert judgement.

The aim is not to demand human approval of every formal object. It is to spend human attention where it has the most leverage: confirming that the candidate statement preserves the intended question and the domain-specific meaning before a proof is treated as a research result.

9. A proposed Phys–Lean workflow

The following is an integration proposal. It does not imply that the repositories currently have a built-in Phys–Lean bridge, a common manifest schema, or automatic Lean export. Each stage would need a deliberately scoped implementation and acceptance tests.

Stage 1 — Source-grounded research package in Phys

Begin with a Type-A Phys package for a paper-scale derivation, or a Type-B package for a textbook or benchmark problem. Pin the source and version; record the relevant equations and locations; identify assumptions, conventions, interpretation, regime, and requested result. Conduct a capability audit against the live APIs and rule registry. Record what is represented, what is executable, what remains unsupported, and why.

Stage 2 — Human-reviewed formalization brief

Produce a short bridge artifact that identifies the exact claim to formalize, the proposed mathematical definitions, the scope and quantifiers, premises, unresolved interpretation choices, and the source passages each item is intended to capture. The brief should preserve the distinction between “the paper states,” “the researcher infers,” “the model assumes,” and “the system derives.” A domain expert or appropriately qualified reviewer checks the fidelity of the formalization before the proof task is treated as settled.

Stage 3 — Lean statement and proof attempt

Translate the reviewed statement into Lean. Reuse Mathlib where suitable, and add new definitions only when the research concept requires them. Let agents search for lemmas, create proof attempts, and iterate with Lean's compiler and tactic feedback. Keep experimental branches and unsuccessful proof attempts when they contain diagnostic information, but do not confuse a large search log with an established result.

Stage 4 — Machine verification and dependency audit

Require a successful Lean build for the relevant module, inspect the theorem's axioms and dependencies, and use additional proof validation steps appropriate to the risk. Lean's documentation explains both the normal kernel-checking guarantee and extra checks for dependencies and potentially misleading proof artifacts [2]. Record exact toolchain, package revisions, theorem names, assumptions, axioms, imports, build outputs, and verifier results. The result is “proved under these formal foundations and dependencies,” not “the physical universe has been proved to obey this theory.”

Stage 5 — Cross-layer conformance and adversarial review

Check that the formal statement corresponds to the reviewed Phys brief; that source equations, definitions, and conditions have not been lost; that all claims of infrastructure usage map to real calls and tests; and that registered gaps remain visible. Perform negative tests: change a sign, drop a necessary condition, swap variable identities, remove a premise, substitute an incorrect intermediate result, or weaken a quantifier. Confirm that the intended gate detects the mutation. A test that merely fails for some unrelated reason is not enough.

Stage 6 — Research and empirical interpretation

Relate the formal result back to the paper's physical framework. If the result generates a measurable prediction, record it as a prediction and state what observations could support or falsify it. Populate empirical status only from actual evidence. If there is no empirical evidence, state that plainly. If multiple hypotheses imply the same current observations, record the underdetermination rather than selecting the most elegant one by preference.

Stage 7 — Preserve a two-way audit trail

The final research package should allow a reader to move from the paper to the Phys representation, from the representation to the Lean theorem, from the theorem to its proof dependencies and axioms, and from the proof back to the original assumptions and interpretation. A change to an assumption or formal definition should be traceable to the proofs and claims that depend on it. If a proof breaks after a change, that is useful evidence; if a proof still passes, it does not automatically mean the research conclusion is unchanged in the relevant informal sense.

10. A potential interoperability contract

A bridge should not be a vague instruction to “use Lean when possible.” It should have explicit records. A minimal future interop record could contain:

- **Research identity:** artifact reference, package type, source citation, exact version or digest, equation/section scope.
- **Scientific context:** framework, interpretation, regime, conventions, source-stated assumptions, inherited assumptions, inferred assumptions, unresolved assumptions.
- **Claim identity:** stable claim ID, informal statement, definitions, requested quantifiers and scope, claim status, alternate readings.
- **Mapping:** Phys mathematical artifact references and corresponding Lean definition/theorem names; an explicit mapping rationale for each critical quantity and condition.
- **Lean environment:** Lean toolchain revision, Mathlib revision, imports, custom definitions, axioms, and theorem identifiers.
- **Verification evidence:** build command, successful/failed result, relevant `#print axioms` output, optional independent checker result, and proof artifact identity.
- **Fidelity review:** reviewer decision or status, issues found, whether independent formalizations were compared, remaining semantic ambiguity.
- **Physical/evidence status:** formally derived, interpreted under a named framework, prediction proposed, empirically tested, supported, constrained, falsified, or not yet observed—with each status defined by the relevant authority rather than invented ad hoc.
- **Limitations and gaps:** unsupported transformation, solver-local computation, missing formal library, unresolved physical assumption, and next justified step.

This schema is a proposal, not an existing Phys specification. It should be introduced only after reviewing both projects' current conventions and authority boundaries. It should not force a single universal “verified” field across logical, source-fidelity, physical-interpretation, and empirical statuses. These are distinct dimensions and should remain distinct.

11. Example: the Bohmian minisuperspace derivation

A useful pilot could start with the existing PhysLib Bohmian `minisuperspace` fixture, which is based on a bounded section of the Dürr–Struyve Quantum Einstein Equations paper. The first task would not be to claim that the entire paper is already formalized. It would be to select one well-scoped relation or derivation slice and inspect exactly what the fixture represents and what its current tests establish.

Phys would be used to pin the paper and equations, map the quantities, document the physical assumptions and reduction, identify which symbolic operations actually execute, record the evidence path, and mark limitations. A reviewer would select a specific theorem-level or identity-level proposition for Lean formalization only after agreeing on its intended meaning, definitions, scope, and assumptions. Lean would then be used to check the formal statement and proof. The final audit would compare the Lean statement to the source map and the Phys artifact, examine axioms and imports, test nearby variations, and keep all missing physical or empirical evidence visible.

The pilot succeeds even if it exposes a gap instead of producing a proof. For example, it might show that the source statement depends on a convention not yet represented, or that a mathematical transformation needed by the derivation does not exist in the current PhysMath registry. That finding would narrow the next implementation task and stop the project from presenting a plausible but unsupported result. A good research infrastructure makes failure informative.

No such direct integration should be advertised as complete until the bridge, builds, mappings, and acceptance tests actually exist and have been run. The pilot is proposed because the existing fixture provides a concrete starting point, not because all dependencies are already solved.

12. Strengths, weaknesses, and boundaries—without making a contest of it

Where Lean is unusually strong

Lean provides a rigorous formal environment for stating propositions and checking proofs. Its kernel-centered architecture, extensive Mathlib ecosystem, explicit declarations, proof dependencies, axioms, and reproducible build workflow are strong foundations for the derivational layer. When an AI proposes a proof, Lean can turn “this appears logically valid” into a mechanically checkable result relative to the chosen formalization.

Where Lean needs complementary process

Lean does not choose the scientifically important question for the researcher. It does not automatically determine whether a formal definition preserves a physical concept from an informal paper. It does not decide which assumptions are justified by an experiment, whether the formal model is a good representation of Nature, or whether competing hypotheses are being compared fairly. These are not inherent shortcomings of the kernel; they are responsibilities beyond the theorem-checking question itself. They need explicit workflows, source context, independent review, and empirical methods.

Where Phys can add leverage

Phys makes the surrounding research context central: source and version, quantity mapping, conventions, assumptions, physical framework, operation capability, artifact identity, evidence, and known gaps. The project has already learned from concrete failure modes that library imports can be cosmetic, tests can miss the intended physics, and a generated proof or derivation can pass a superficial acceptance gate while failing to establish the source claim. Making API adoption and testing claims executable is one response to those lessons.

Where Phys must remain modest

Phys is not yet a general proof assistant, a complete symbolic mathematics engine, an integrated experimental pipeline, or an autonomous hypothesis-discovery system. Some object families and operations are representable without being executable. Numerical solvers and numerical integration are outside the current representation-layer scope. Unit metadata is not a full dimensional-semantics system. Current protocol documents define rigorous expectations, but documentation is not evidence that every future package complies; real implementation, tests, and reviewer evidence are still required.

Shared limitations

Both systems depend on the quality of their definitions and on the legitimacy of imported knowledge. Both can be embedded in bad processes. Both need version control, dependency audits, testing, review, and clear claims. Neither can transform a human preference into an empirical fact, nor infer physical truth from internal consistency alone. The right strategy is not to demand that either system do everything, but to connect them with explicit contracts and preserve the boundary between their guarantees.

13. Evaluation should measure the whole chain

A meaningful Phys–Lean benchmark should not score only whether a model produced compilable Lean or a correct-looking final answer. It should separately evaluate the steps that turn research material into a justified conclusion.

Source fidelity: Did the agent use the correct paper/version/equations and preserve the author's stated scope? Was source evidence attached to each core claim?

Assumption fidelity: Were assumptions classified correctly, propagated where required, and neither silently added nor removed? Were regime and convention changes recorded?

Formal-statement fidelity: Does the formal statement correspond to the intended informal claim? Does it avoid triviality, circularity, accidental weakening, and scope drift? Was the mapping reviewed independently of the proof search?

Proof validity: Does Lean accept the proof of the declared statement? What axioms and imported results does it depend on? Does the proof compile under a pinned toolchain?

Derivation traceability: Can every important step be traced to premises and operations? Did tests independently check key results and reject selected plausible mutations?

Hypothesis quality: Does the proposed hypothesis make a clear distinction from alternatives? Are there meaningful potential counterexamples or discriminating predictions? This will often require expert assessment rather than a simple automated score.

Empirical connection: Is there actual evidence, and does it support, constrain, or falsify the model within a specified regime? If there is no relevant evidence, is the status explicitly unobserved or unavailable?

Honest failure handling: When the required capability or information is missing, does the system report the gap, or does it silently substitute an easier task? Does a failed attempt preserve enough information to guide the next step?

Reproducibility: Can another contributor rebuild the package, inspect exact versions, rerun the checks, and understand the status of the result without trusting the original agent's narrative?

These metrics should not be collapsed into a single number too early. A package can have excellent proof validity and poor source fidelity, or excellent source provenance and no complete proof. The dashboard should preserve this profile rather than average away a critical failure.

14. Why the combination matters in the larger AI research ecosystem

The long-term significance of this work is not that every scientific paper must be converted into Lean or that every physics result should be routed through a single toolchain. The significance is the possibility of a shared, layered research workflow in which AI agents can search and propose, source-grounded representations make their choices inspectable, formal proof systems check the mathematical claims they are suited to check, and empirical science remains the authority for questions that require observation of the world.

That architecture could help universities preserve a trail from a lecture problem to its assumptions and derivation; help research teams review AI-assisted formalizations; help tool builders state exactly what a verification badge means; and help the public distinguish an AI's confident explanation from a supported conclusion. It may also make scientific collaboration more modular: an expert can check physical interpretation, a formal-methods researcher can review a Lean theorem, a software engineer can strengthen validators, and an experimenter can assess predictions. Each role sees a concrete artifact and a clearly bounded responsibility.

This is especially important as agents become more autonomous. If the main task were only to find proofs, a proof assistant could be the center of the story. But AI research agents are increasingly asked to select papers, formulate claims, invent abstractions, propose mechanisms, and decide which experiments or calculations to try. These upstream choices shape every downstream result. The more autonomy we grant, the more important it becomes

to record what the agent chose, why, from which sources, under which assumptions, with what uncertainty, and what evidence was generated.

The end state should not be a system that says “the AI verified it” and expects everyone to agree. It should be a research workspace that says: “Here is the source claim; here is the formal statement we believe captures it; here is who or what reviewed that mapping; here is the proof Lean checked; here are the assumptions and axioms; here are the operations the infrastructure actually executed; here is what remains unverified; and here is the experiment that would help us learn whether the model describes the world.”

That is a far more useful promise, and a far more demanding one.

15. Collaboration proposal and next steps

A practical collaboration should begin with a narrow, inspectable pilot rather than a broad claim of complete integration.

1. **Select one bounded derivation slice** from the existing Bohmian minisuperspace fixture or another suitable PhysLib example. Pin the source and identify the exact claim and equations.
2. **Complete the source and capability audit in Phys.** Confirm every source link, assumption, definition, operation, and limitation. Treat unsupported steps as gaps, not as proof obligations that can be waved away.
3. **Draft a small formalization brief.** Define the target Lean proposition and its intended meaning. Record ambiguous readings and assumptions. Obtain human review of source fidelity before declaring the target fixed.
4. **Create a Lean proof or a diagnostic failure.** Use Mathlib where suitable. Build and run the proof checks under pinned versions. Audit theorem dependencies and axioms. Do not weaken or rewrite the claim silently if proof search fails.
5. **Build a cross-layer acceptance suite.** Check source-to-formal mapping, assumptions, proof status, mutation detection, and gap reporting. Require a reviewer to adjudicate the theorem's intended meaning independently from proof success.
6. **Publish a reproducible case study.** Include the paper citation and scope, Phys package, Lean files, versions, exact commands, test results, assumptions, reviewer findings, and known limits.
7. **Expand only from evidence.** The pilot will reveal whether the biggest missing piece is source-mapping tooling, physical semantics, a mathematical operation, a formalization template, Lean library coverage, or review methodology. Prioritize the smallest reusable improvement supported by that finding.

Academia can contribute paper-level review and formalization judgement. The Lean community can advise on the proper use of Lean, Mathlib, and theorem dependencies. Industry can support reproducible infrastructure, CI, and agent tool integrations. Open-source contributors

can build validators, adapters, tests, and tutorials. Sponsorship is valuable because sustained maintenance, independent review, and high-quality documentation take real time, even when the core software is freely available.

Conclusion: make the claim traceable before calling the proof trustworthy

Phys and Lean are not rival answers to the same question. They are complementary tools for different parts of a harder problem: getting from a human or AI-generated idea to a conclusion that deserves confidence.

Lean's contribution is unusually strong when the proposition is precise: it can check that a formal proof follows under the chosen definitions, theorems, and axioms. Phys's proposed contribution is upstream and contextual: it helps maintain the identity, source relationship, physical assumptions, interpretation, capability boundaries, and evidence trail around the mathematical work. Those boundaries matter because a proof can be valid for the wrong formal statement, and a faithful research record can still contain an invalid derivation. Both failures must be addressed.

The larger challenge is hypothesis formulation—deciding what to ask, how to express it, which premises to accept, which alternatives to compare, and what evidence could change our minds. AI can help explore this space with increasing autonomy. Humans remain essential for meaning, scientific judgement, and accountability. Formal tools should make the work more checkable, not confer authority on a mistaken premise. Empirical methods remain essential for claims about the physical world.

The opportunity for Phys is therefore immense not because it has already solved these problems, but because it is targeting a neglected part of the chain that becomes more important as AI agents take on more research work: the fidelity of what is being asked and represented in the first place. Paired with mature formal verification, this approach could help build research systems that are more transparent, more adversarial toward their own mistakes, and more useful to both experts and the public.

The invitation is to build that future together—carefully, openly, and with the humility to let a well-documented failure teach us as much as a successful proof.

References and further reading

[1] Lean Language Reference, “Introduction” and kernel/tactic architecture. https://lean-lang.org/doc/reference/latest/

[2] Lean Language Reference, “Validating a Lean Proof.” Includes the distinction between whether a theorem has a valid proof and what its statement means, plus guidance on inspecting axioms and dependencies. https://lean-lang.org/doc/reference/latest/ValidatingProofs/

[3] Lean, “Mathlib: A Foundation for Formal Mathematics Research and Verification.” Overview of Mathlib's coverage, composability, maintenance, and uses. https://lean-lang.org/use-cases/mathlib/

[4] Phys project, `PHYSMATH_PHYS_EXTERNAL_ARCHITECTURE_v1.5.md` and Phase-1 representation architecture. Project architecture documents govern ownership and dependency boundaries, no automatic inference of physical or empirical truth, and no generic theorem prover inside PhysMath.

[5] PhysTrace project, `DERIVATION_SPEC_GUIDELINES.md`. Governs source identity, quantity/source maps, assumptions, capability audits, independent expected results, mutation testing, provenance, and status discipline.

[6] Phys project, `phys-demo/PROTOCOL.md` and `phys-demo/adr_package_types.md`. Define the Type-A formalization and Type-B benchmark-solution workflows and their anti-evasion and adoption requirements.

[7] Auguste Poiroux, Gail Weiss, Viktor Kunčak, and Antoine Bosselut, “Reliable Evaluation and Benchmarks for Statement Autoformalization,” *Proceedings of EMNLP 2025*, pp. 17947–17969. https://aclanthology.org/2025.emnlp-main.907/ DOI: https://doi.org/10.18653/v1/2025.emnlp-main.907

[8] Arshia Soltani Moakhar, Iman Gholami, Max Springer, Mahdi Jafari-Raviz, and MohammadTaghi Hajiaghayi, “Beyond the Library: An Agentic Framework for Autoformalizing Research Mathematics,” posted June 2026. https://arxiv.org/abs/2606.31134

[9] Phys infrastructure repositories: PhysArtifact https://github.com/PithomLabs/phys-artifact ; PhysMath https://github.com/PithomLabs/phys-math ; PhysLib https://github.com/PithomLabs/phys-lib ; PhysTrace https://github.com/PithomLabs/phys-trace .